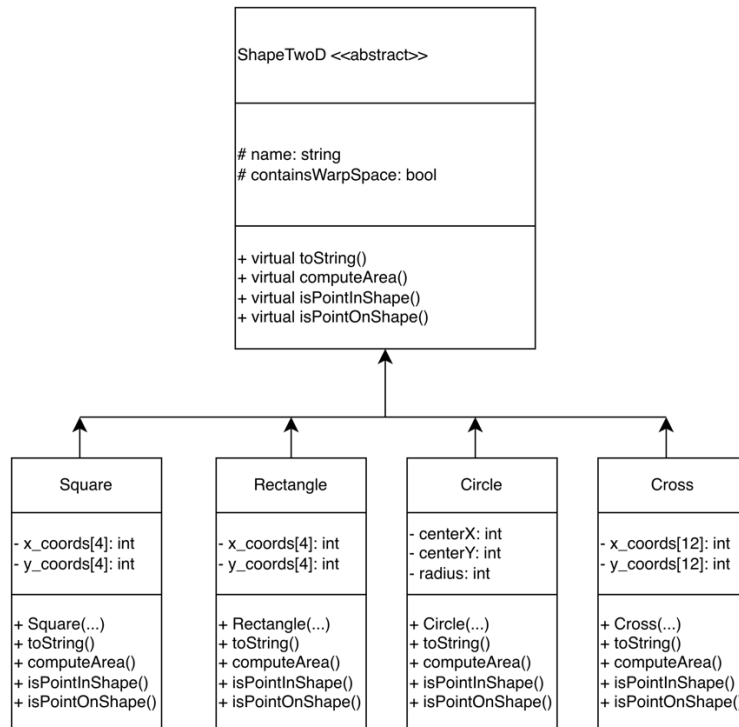


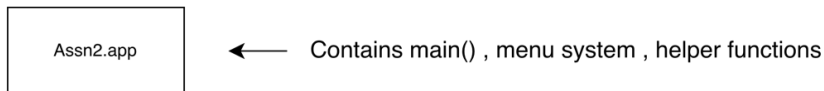
Part 1. Overview Diagram

1. Class Hierarchy

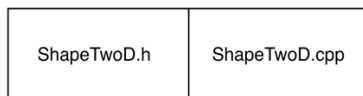


2. File Structure

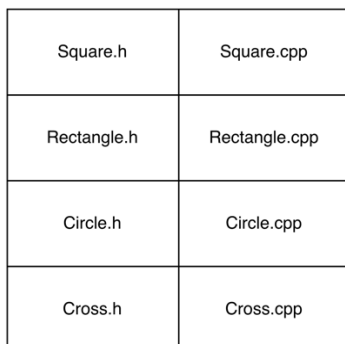
Main Driver:



Base Class:



Derived Classes:



3. Key Functions in Assn2.cpp

Helper Functions:

- toLowerCase() – Convert string to lowercase
- toProperCase() – Capitalize first letter only
(Used for case – insensitive input handling)

Menu Functions:

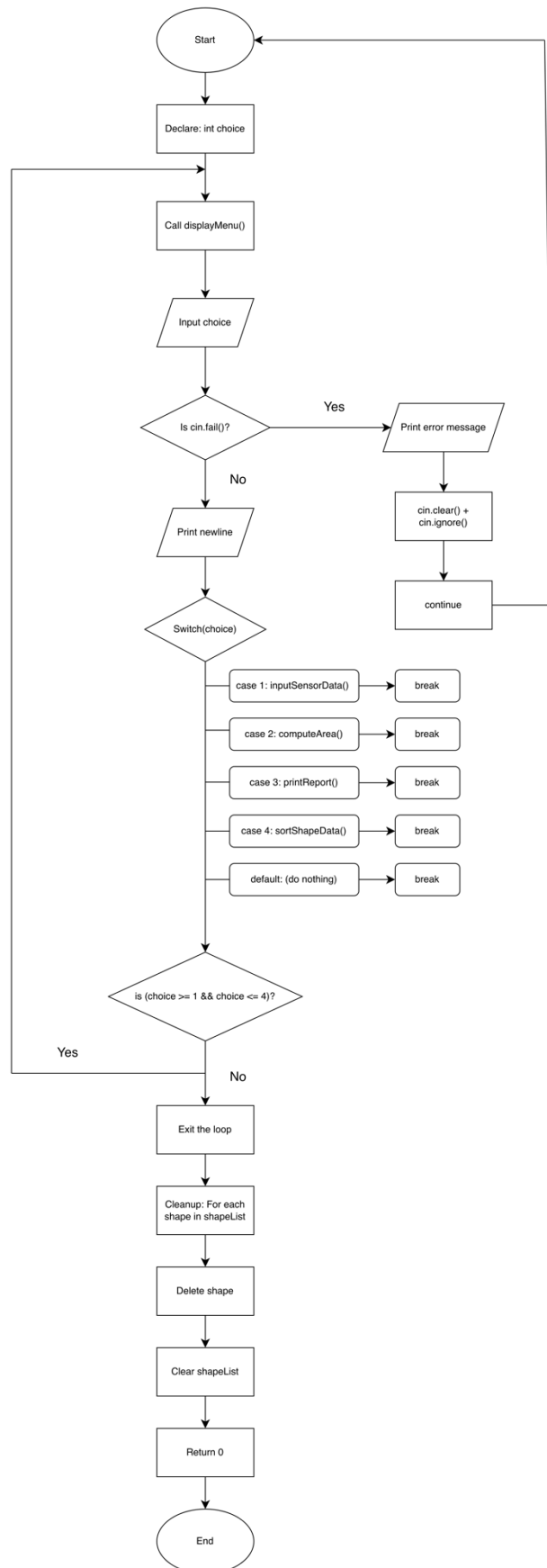
- displayMenu() - Display 4 menu options (1-4)
- inputSensorData() - Option 1: Input shape data → Includes case conversion and validation
- computeArea() - Option 2: Confirm area computation
- printReport() - Option 3: Display all shape details
- sortShapeData() - Option 4: Sort shapes (a/b/c options)

Main Function:

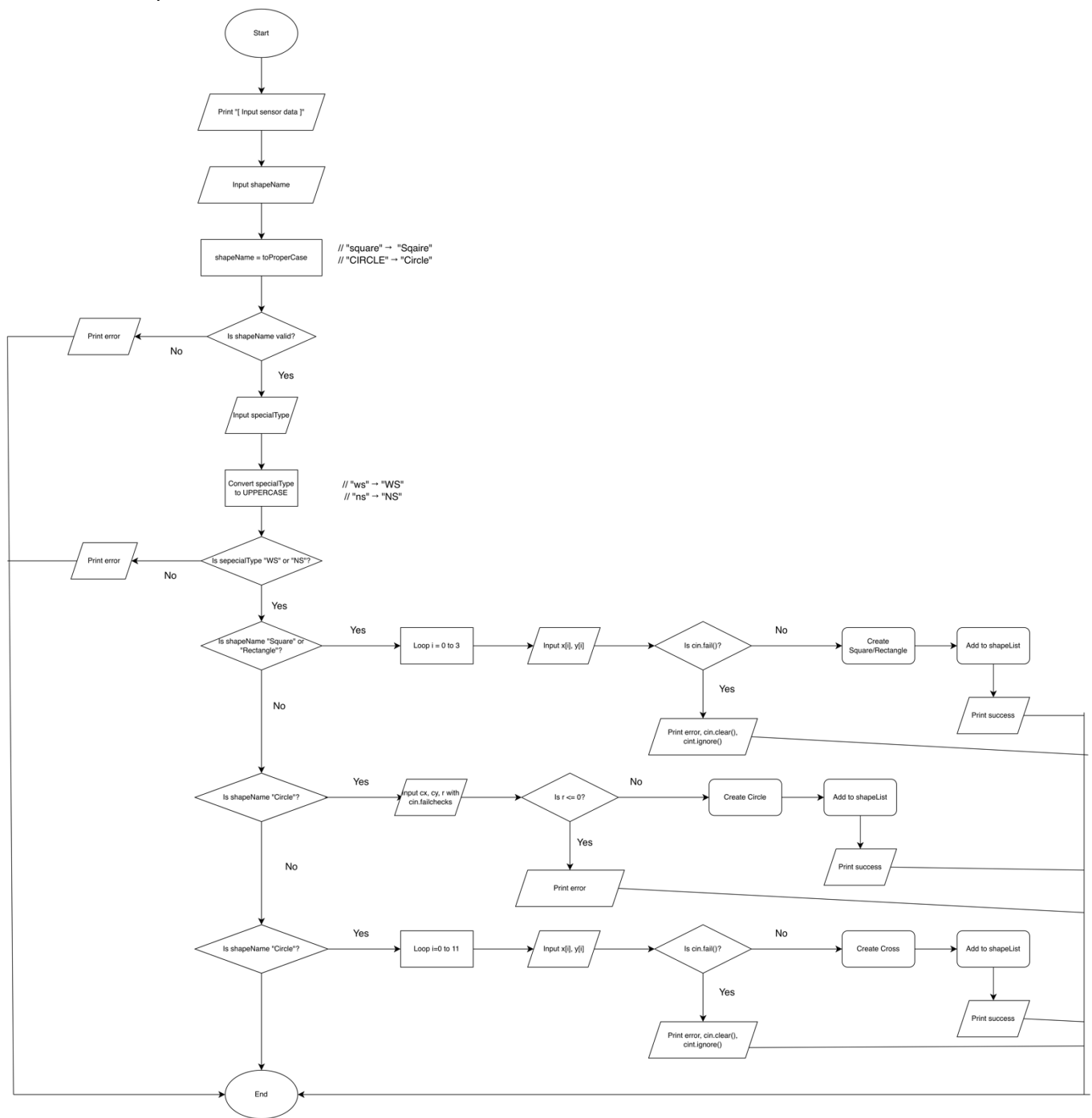
- main() - Program entry point
 - Loop continues while choice is 1-4
 - Exits when choice < 1 or choice > 4
 - Includes cin.fail() validation
 - Cleanup: delete all shapes before exit

Part 2: Flowchart

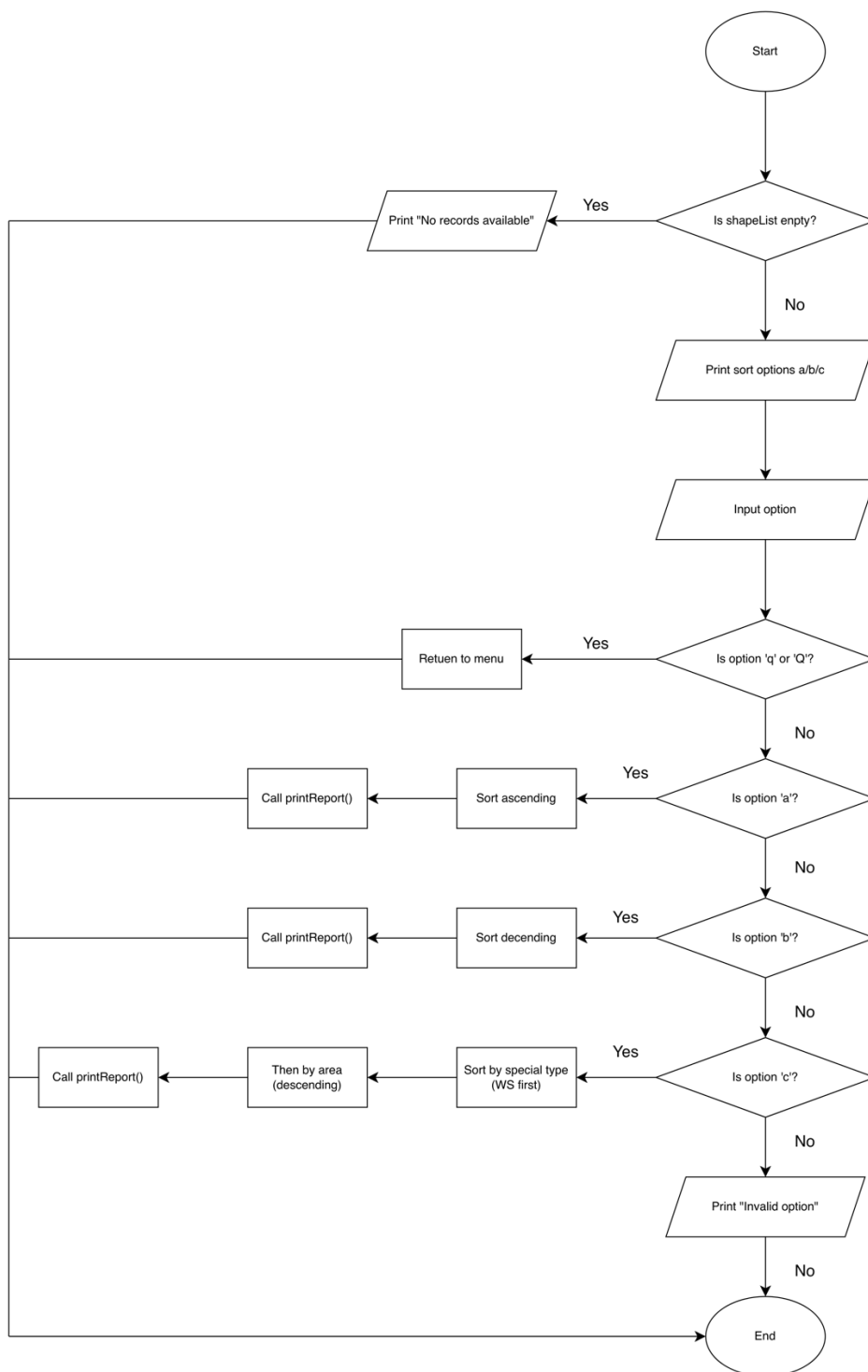
Flowchart 1: main() Function



Flowchart 2: inputSensorData



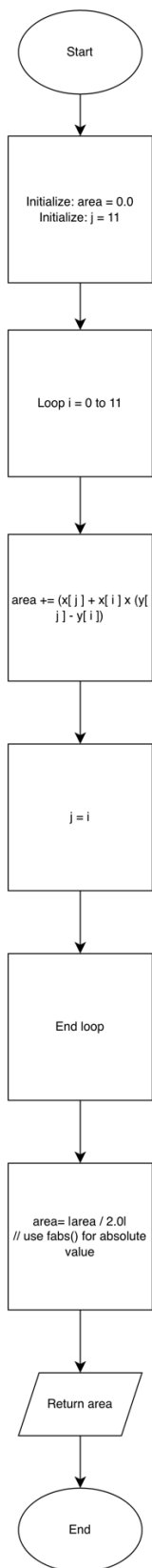
Flowchart 3: sortShapeData()



Sort Option C - Special Type and Area:

1. Primary sort: containsWarpSpace (WS=true, NS=false) → WS shapes appear first
2. Secondary sort: area (descending within each group) → Larger areas appear first

Flowchart 4: Cross::computeArea() – shoelace Formula



Shoelace Formula (Gauss's Area Formula)

Purpose: Calculate area of any polygon using vertex coordinates

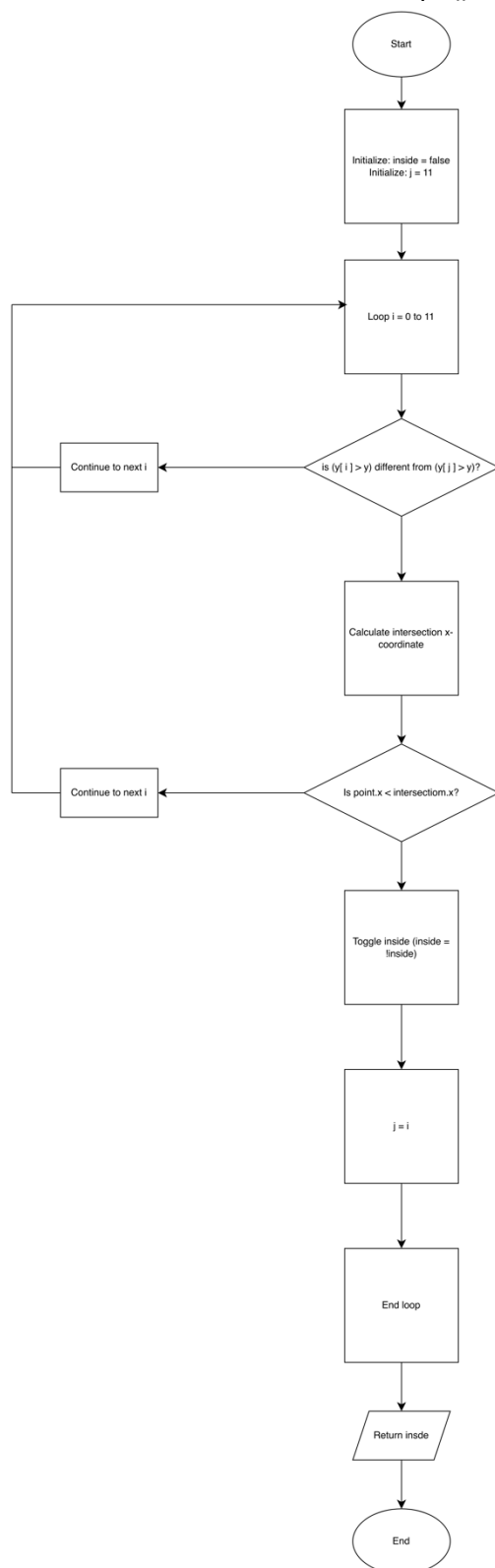
Formula: $A = \frac{1}{2} |\sum (x[i] \times y[i+1] - x[i+1] \times y[i])|$

Why it works:

- Treats polygon as sum of trapezoids
- Positive areas cancel negative areas
- Works for any simple (non-self-intersecting) polygon

Time Complexity: $O(n)$ where n = number of vertices (12 for Cross)

Flowchart 5: Cross:: isPointInShape() – Ray casting



Ray Casting Algorithm

Purpose: Determine if a point is inside a polygon

Concept:

1. Cast a horizontal ray from the point to infinity ($\rightarrow$)
2. Count how many polygon edges the ray crosses
3. Odd number of crossings $\rightarrow$ point is INSIDE Even number of crossings $\rightarrow$ point is OUTSIDE

Why it works:

- Every time the ray crosses an edge, it alternates between inside and outside the polygon
- Starting from outside (even=0), first crossing makes it inside (odd=1)

Time Complexity: $O(n)$ where n = number of edges

Part 3: Algorithm Descriptions

1. Case-Insensitive Input Handling

Implementation:

- toLowerCase(): Converts entire string to lowercase using tolower()
- toProperCase(): Converts to lowercase first, then capitalizes first letter
- Applied to: shape names and special types before validation

Benefits:

- Improves user experience (no need for exact case)
- Demonstrates defensive programming
- Still maintains proper formatting in output

2. Input Validation Strategy

Multi-layer Validation:

1. Type validation: cin.fail() catches non-integer inputs
2. Range validation: radius > 0 for circles
3. Value validation: shape names and special types checked against allowed values

Error Recovery:

- cin.clear(): Reset error state
- cin.ignore(10000, '\n'): Discard invalid input from buffer
- return: Return to main menu instead of crashing

3. Memory Management

Allocation:

- Shapes created with 'new' and stored in vector<ShapeTwoD*>
- Enables polymorphism (base class pointers to derived objects)

Cleanup:

- Before program exit, loop through shapeList
- Delete each shape object
- Clear the vector
- Prevents memory leaks